



FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

UNIVERSIDAD DE LAS AMERICAS

Integración de Sistemas

EXAMEN PRACTICO PROGRESO 2

Caso: Integración de Sistemas - Salud360

Nombre del Estudiante:

Gabriel Padilla

Fecha: 2026-06-04

Modalidad: Individual, presencial



FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

REPOSITORIO GITHUB DE LA SOLUCIÓN

Repositorio GitHub de la solución: <https://github.com/Gabriel2146/progreso2-integracion-padilla-gabriel>

El repositorio es público y contiene todo el código fuente necesario para ejecutar la solución. Incluye el Dockerfile, docker-compose.yml, código fuente Java y los archivos de datos de salida, adicional el Readme.md.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

1. IDENTIFICACIÓN DEL PROBLEMA DE INTEGRACIÓN

Problema principal

Salud360 administra una red de centros médicos con múltiples sistemas que no están integrados. Cuando se registra una cita médica, los datos deben copiarse manualmente entre cinco sistemas independientes: Agenda Medica, Facturación, Notificaciones, Analítica y el sistema legado de Auditoria. Este proceso manual genera errores, retrasos, registros duplicados y falta de trazabilidad.

Sistemas que deben integrarse

Sistema	Rol en la integración
Agenda Medica	Origen: registra solicitudes de cita y dispara el flujo
Facturación	Consumidor único: genera orden de cobro por cada cita confirmada
Notificaciones	Consumidor de evento: envia mensajes al paciente
Analítica	Consumidor de evento: alimenta indicadores operativos
Auditoria (legado)	Sin API ni mensajería: consume archivos CSV desde carpeta compartida

Datos que deben circular

- idCita, paciente, correo, especialidad, fechaCita, sede, valor
- Mensaje a Facturacion: idCita + paciente + especialidad + valor + tipoMensaje
- Evento a Notificaciones/Analítica: todos los campos + tipoEvento = CITA_CONFIRMADA
- Registro CSV para Auditoria: línea con todos los campos en formato plano

Riesgos si la integración se mantiene manual

- Duplicación de registros: la misma cita puede enviarse múltiples veces a Facturacion, generando cobros duplicados.
- Inconsistencia de datos: distintos sistemas pueden tener versiones diferentes de la misma cita.
- Falta de trazabilidad: sin registro automático no hay auditoria del estado de cada cita.
- Retrasos operativos: el proceso manual depende de la disponibilidad del personal.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

- Riesgo de omisión: un sistema puede no recibir la información si la persona olvida actualizar.

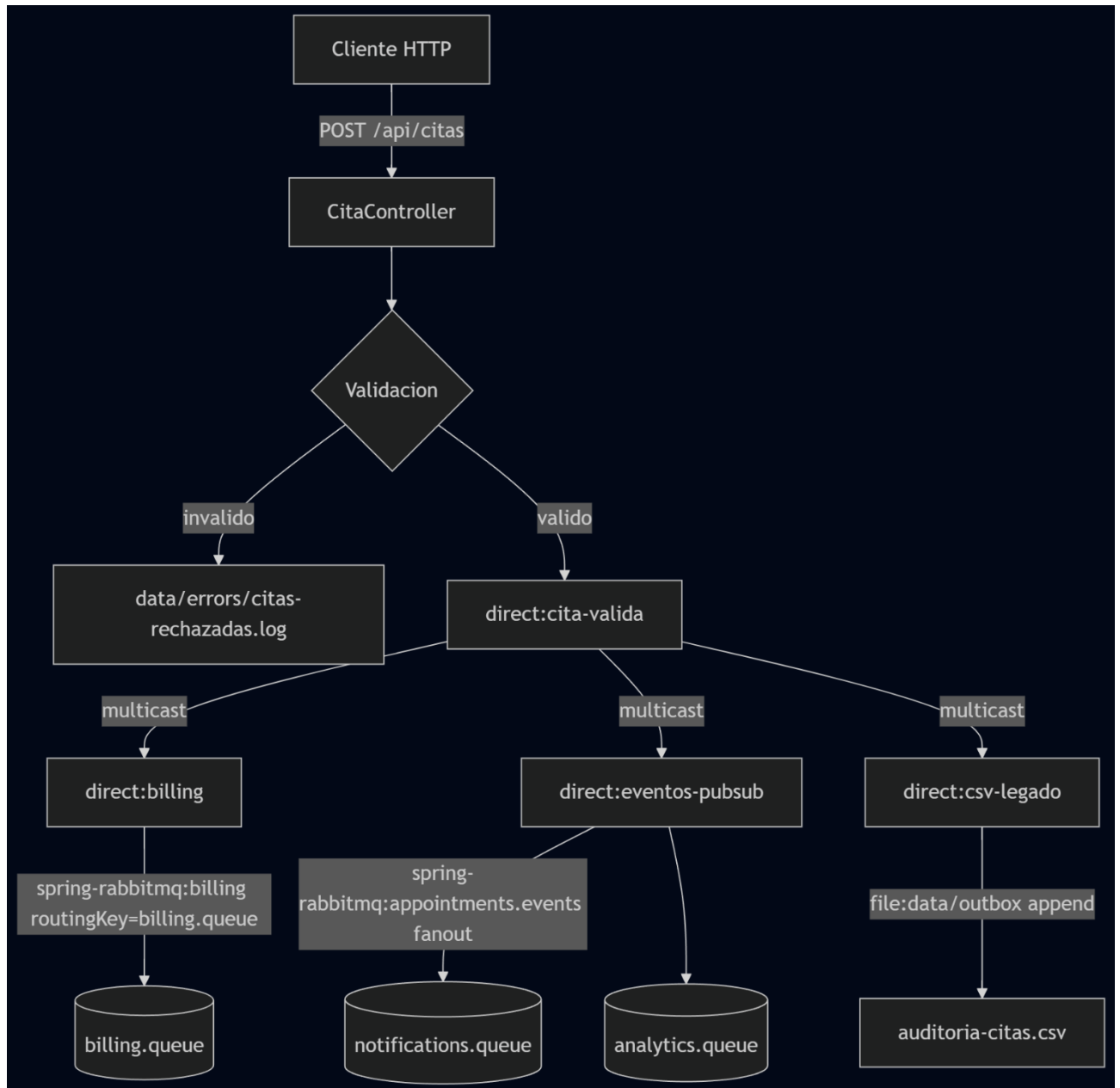
2. ANÁLISIS DE ESTILOS DE INTEGRACIÓN

Necesidad del caso	Patron aplicado	Justificación
Exponer el registro de citas	API REST	Permite interoperabilidad sin acoplamiento. Cualquier cliente HTTP puede invocarla. El contrato es el JSON del payload.
Enviar solicitud a Facturacion	Point-to-Point Channel	La orden de cobro debe generarse exactamente una vez. Si varios consumidores recibieran el mensaje se crearían ordenes duplicadas. El patrón P2P garantiza un único consumidor.
Notificar a varios sistemas	Publish/Subscribe Channel	Notificaciones y Analítica deben reaccionar al mismo evento de forma independiente. El exchange fanout entrega una copia a cada cola sin que los consumidores se conozcan entre sí.
Enviar datos al sistema legado	Transferencia de archivos (CSV)	El sistema de Auditoria no tiene API ni conexión a mensajería. La única forma de integrarlo es mediante un archivo CSV en una carpeta compartida, que la aplicación genera y escribe automáticamente.

3. DISEÑO DE LA SOLUCIÓN

Diagrama de flujo

El siguiente diagrama describe el flujo completo desde la recepción de la cita hasta la distribución a todos los sistemas destino:



Descripción de componentes

Componente	Responsabilidad
CitaController	Expone POST /api/citas. Valida campos, delega al servicio de validación y dispara el flujo Camel vía Producer Témplate.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

CitaValidationService	Valida los 7 campos obligatorios. Escribe el log de errores con timestamp, idCita, motivos y payload.
CitaIntegrationRoute	Define las 5 rutas Camel: principal (multicast), facturación P2P, pubsub fanout, CSV legado, y DLQ de errores.
RabbitMQConfig	Declara con Spring AMQP: billing.queue, notifications.queue, analytics.queue, exchanges billing (direct) y appointments.events (fanout), y sus bindings.
RabbitMQ (Docker)	Bróker de mensajería. Gestiona las colas y exchanges. Accesible vía panel web en http://localhost:15672 .

Descripción de rutas de integración

- ruta-principal (direct:cita-valida): recibe la cita valida y hace multicast paralelo a las 3 rutas destino.
- ruta-facturacion-p2p (direct:billing): serializa un comando JSON y lo envía al exchange direct 'billing' con routingKey='billing.queue'. Patron Point-to-Point.
- ruta-pubsub-eventos (direct:eventos-pubsub): serializa el evento CITA_CONFIRMADA y lo publica en el exchange fanout 'appointments.events'. El broker lo entrega a notifications.queue y analytics.queue simultáneamente. Patron Publish/Subscribe.
- ruta-csv-auditoria (direct:csv-legado): genera una línea CSV con Locale.US (punto decimal) y la escribe con el componente file de Camel en modo Append.
- ruta-error-camel (direct:manejo-error-camel): Dead Letter Channel con 3 reintentos y 2s de delay. Registra el fallo sin detener la aplicación.

4. EVIDENCIA DE IMPLEMENTACIÓN

1. Aplicación iniciada sin errores

La aplicación levanta con Spring Boot 3.2.5 + Apache Camel 4.4.4 conectado a RabbitMQ (vía Docker). Las 5 rutas Camel se inician correctamente:

```
Routes startup (started:5)

Started ruta-principal          (direct://cita-valida)
Started ruta-facturacion-p2p    (direct://billing)
Started ruta-pubsub-eventos     (direct://eventos-pubsub)
Started ruta-csv-auditoria      (direct://csv-legado)
Started ruta-error-camel        (direct://manejo-error-camel)

Apache Camel 4.4.4 started in 39ms
```

2 y 3. Request valido y respuesta exitosa

Request enviado a POST <http://localhost:8080/api/citas>:

```
{ "idCita": "CITA-1001", "paciente": "Ana Torres",
  "correo": "ana.torres@email.com", "especialidad": "Cardiologia",
  "fechaCita": "2026-06-15", "sede": "Centro Norte", "valor": 45.50 }
```

Respuesta HTTP 200 OK:

```
{
  "estado": "REGISTRADA",
  "mensaje": "Cita registrada y distribuida correctamente",
  "idCita": "CITA-1001"
}
```

4 y 5. Mensajes en colas RabbitMQ

Verificación vía API del panel de administración (<http://localhost:15672>) con Invoke-RequestMethod:

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Cola	Mensajes	Mensajes listos	Consumidores
billing.queue	2	2	0
notifications.queue	2	2	0
analytics.queue	2	2	0

Las 3 colas tienen mensajes confirmando que: billing.queue recibió vía P2P, y notifications.queue + analytics.queue recibieron vía PubSub (fanout).

6. Archivo CSV generado

Ruta: data/outbox/auditoria-citas.csv (generado automáticamente por la ruta-csv-auditoria de Camel):

```
idCita,paciente,correo,especialidad,fechaCita,sede,valor
CITA-1001,Ana          Torres,ana.torres@email.com,Cardiologia,2026-06-15,Centro
Norte,45.50
```

El valor 45.50 usa punto decimal (Locale.US) para mantener el formato CSV correcto independientemente del sistema operativo.

7. Registro de error ante solicitud invalida

Request invalido enviado: solo el campo 'paciente', omitiendo los 6 restantes.

Respuesta HTTP 400 Bad Request:

```
{ "estado": "RECHAZADA",
  "errores": ["idCita es obligatorio", "correo es obligatorio",
              "especialidad es obligatoria", "fechaCita es obligatoria",
              "sede es obligatoria", "valor debe ser mayor a 0"] }
```

Registro en data/errors/citas-rechazadas.log:

```
2026-06-04 20:18:36 | idCita: N/A | Motivos: idCita es obligatorio,
correo es obligatorio, especialidad es obligatoria, fechaCita es obligatoria,
sede es obligatoria, valor debe ser mayor a 0 |
```


FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

```
Payload: {"idCita":null,"paciente":"Ana Torres","correo":null,...}
```

5. REFLEXIÓN TÉCNICA FINAL

1. Por qué no sería suficiente resolver todo únicamente con archivos?

Los archivos son asíncronos por naturaleza y no garantizan entrega en tiempo real. Si el sistema de Facturación requiere procesar la cita inmediatamente para generar la orden de cobro, una integración por archivos podría introducir retrasos significativos (dependiendo del intervalo de polling). Además, los archivos no proveen notificación de lectura, manejo de reintentos, ni exclusión mutua nativa: dos procesos podrían leer el mismo archivo simultáneamente y generar cobros duplicados. La mensajería (RabbitMQ) resuelve estos problemas con colas durables, confirmación de mensajes (ACK/NACK) y entrega garantizada.

2. Por qué no sería adecuado enviar la facturación por Publish/Subscribe?

El patrón Publish/Subscribe entrega una copia del mensaje a TODOS los suscriptores. Si el sistema de Facturación estuviera suscrito junto a Notificaciones y Analítica al mismo exchange fanout, y en algún momento hubiera múltiples instancias del sistema de Facturación activas (escalado horizontal), cada instancia recibiría el mensaje y generaría una orden de cobro independiente, produciendo facturas duplicadas. El patrón Point-to-Point garantiza que el mensaje es consumido por exactamente un receptor, lo cual es crítico para transacciones financieras.

3. Que ventaja aporta RabbitMQ frente a una integración directa API contra API?

Con integración directa API-API, si el sistema destino está caído en el momento del registro de la cita, la operación falla y se pierde el evento. RabbitMQ actúa como buffer persistente: el mensaje queda almacenado en la cola (`durable=true`) hasta que el consumidor esté disponible para procesarlo, garantizando entrega eventual. Además, desacopla completamente al productor del consumidor: la API de citas no necesita conocer la dirección



FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ni el estado de Facturación, Notificaciones o Analítica. Esto permite escalar, reiniciar o actualizar cualquier sistema sin afectar al resto.

4. Que mejoraría si esta solución tuviera que operar en producción?

- Seguridad: autenticación en la API (JWT/OAuth2), credenciales RabbitMQ no hardcodeadas (usar secretos de Kubernetes o Vault), TLS en la conexión AMQP.
- Idempotencia: validar que no se procese dos veces el mismo idCita (tabla de citas procesadas en base de datos)
- Observabilidad: métricas con Micrometer + Prometheus, trazas distribuidas con OpenTelemetry, alertas si una cola supera un umbral de mensajes acumulados.
- Alta disponibilidad: RabbitMQ en clúster con nodos de respaldo, la aplicación en múltiples replicas detrás de un load balancer.
- Schema registry: validar el formato de los mensajes con Apache Avro o JSON Schema para evitar mensajes malformados entre sistemas.
- Dead Letter Queue persistente: los mensajes que fallen tras los reintentos deben ir a una DLQ que sea revisada y reintentada manualmente o por proceso automático.